

Technical Assessment Write-up

Jeroen Buil

Task 1: Web (Vue 3): Fast Periodic Visual Stimulation

Approach to get timing right: rAF

Using timers like `setTimeout`/`setInterval` are obviously not the right way to approach a use-case like this. Due to overhead and system usage you simply cannot assure that EXACTLY that the timeout starts and ends when intended and the timeout the stimuli picture is displayed at the correct moment. In practice these timers accumulate, meaning each delay causes a further drift away from the actual moment you wish to display a stimuli; the longer the sequence the larger the drift at the end will be.

Instead I am relying on `requestAnimationFrame` (rAFs), which tell the browser that you wish to perform an action and instruct the browser to do this before the next frame is repainted. Such rAF callbacks are synced to a browser's actual display refresh (typically ~60 Hz / 16.67 ms/frame) and unlike timers mentioned before, they don't accumulate.

rAFs are typically used to reduce the compute needed when nothing is happening in the webapp and can be used e.g. to make animations run smoother (because you only perform them (and their underlying actions as many times as the frames are actually updating). In the use case of a visual experiment, rAFs can also be used to monitor when a frame is updated and display the stimuli at that exact moment (and additionally log when this happens).

Actual implementation:

I am updating displayed stimulus inside a loop that is called '`stimulusUpdateLoop()`'. This loop is called using the `requestAnimationFrame` function executes the loop before the next frame renders and it passes a timestamp.

The function itself does 4 main things:

1. sets the `startTime` on first call => this is used as time reference point for all subsequent rAFs.
2. Updates the `currentIndex` whenever the elapsed time passes into the moment the next stimulus should appear => this triggers Vue to display the next stimulus
3. Logs the onset times in the store => used for the result view to calculate timing stats
4. Updates the `rafId` and schedules a new iteration of this same loop for the next frame render.

|

```
function stimulusUpdateLoop(timestamp) {  
  if (startTime === null) {  
    startTime = timestamp
```

```

    // Stimulus onset logging for first stimulus (index=0) in sequence
    store.onsetLog.push({
      index: 0,
      isOddball: store.sequence[0].isOddball,
      onsetMs: 0,
    })
  }

  const elapsed = timestamp - startTime
  const targetIndex = Math.floor(elapsed / frameIntervalMs) // updates targetIndex when it is time to show
the next image in the sequence

  // Update the currentIndex if the targetIndex updates => this triggers a rerender of the sequence viewer
with the new image
  if (targetIndex > currentIndex.value && targetIndex < store.sequence.length) {
    currentIndex.value = targetIndex
    // Stimulus onset logging:
    store.onsetLog.push({
      index: targetIndex,
      isOddball: store.sequence[targetIndex].isOddball,
      onsetMs: elapsed,
    })
  }

  // if all images in the sequence have been displayed, then move to the results view
  if (targetIndex >= store.sequence.length) {
    store.goTo('results')
    return // sequence done => don't schedule another frame (go straight to onUnmounted instead)
  }

  // update rafId and reschedules this same function for the next frame
  rafId = requestAnimationFrame(stimulusUpdateLoop)
}

// When RunView is mounted store animation frame request id
onMounted(() => {
  rafId = requestAnimationFrame(stimulusUpdateLoop)
})

// When RunView is unmounted cancel animation frame request
onUnmounted(() => {
  if (rafId !== null) cancelAnimationFrame(rafId)
})

```

How accurate is my approach?

The stats below (taken from the Results View in the app after the stimuli are run) show very minimal difference between the mean interval and the target rate, with a small standard deviation (<1ms). This applies to both the random base stimuli as the oddball stimuli (every N-th frame).

Additionally, no dropped stimuli are observed, which would be disastrous for an experiment if the user misses a stimuli because the browser was lagging.

Considering the above, I would **conclude that the stimuli display accuracy is high, but it is not perfect**. Some observations can be made.

- *First stimulus interval higher than the rest?*

The worst case (vs the ideal time) indicates the stimulus that has deviated the furthest from it's ideal time point (in reference to the start of the sequence). This worst case seem to always happen on the interval between the first and the second stimuli. This can likely be attributed to the first time image decoding cost. With ~4-7ms (depending on the run) it is not even 5% deviation from the base rate, but at higher stimulation frequencies this certainly isn't ideal behaviour. Luckily this time can be reduced, see ways to improve: '**Image preloading**.'

- *Minimum interval is smaller than target interval?*

While my approach ensures that a stimuli is never displayed too early, the min interval are still lower than the target interval rate. This may seem counter intuitive, but this is not a bug. This can happen when e.g. one stimulus is delayed and the next one is perfectly on time, causing a delta that is smaller than the target. These variations luckily are small (in the order of 1 std).

<u>Run setup:</u>			
Base rate	6 Hz	Oddball every	5th stimulus
Sequence length	60 s	Stimulus set	fpvs
<u>All intervals:</u>			
Target (6.00 Hz)	166.67 ms	Mean ± std	166.69 ± 0.63 ms
Min	166.00 ms	Max	174.50 ms
Worst case (vs. ideal time)	7.83 ms		
<u>Oddball intervals:</u>			
Target (1.20 Hz)	833.33 ms	Mean ± std	833.32 ± 0.47 ms
Min	833.00 ms	Max	834.00 ms
Worst case (vs. ideal time)	7.83 ms		
<u>Dropped stimuli:</u>			
Total dropped	0	Gaps	0
Min dropped per gap	n/a	Max dropped per gap	n/a

Figure 1: Interval and dropped stimuli stats. Run setup shows the meta data from the current sequence run settings. The intervals are split between All (both base and oddball stimuli) and only the oddball stimuli. Dropped stimuli indicates any dropped stimuli (e.g. due to lag), which did not happen during any of the test runs.

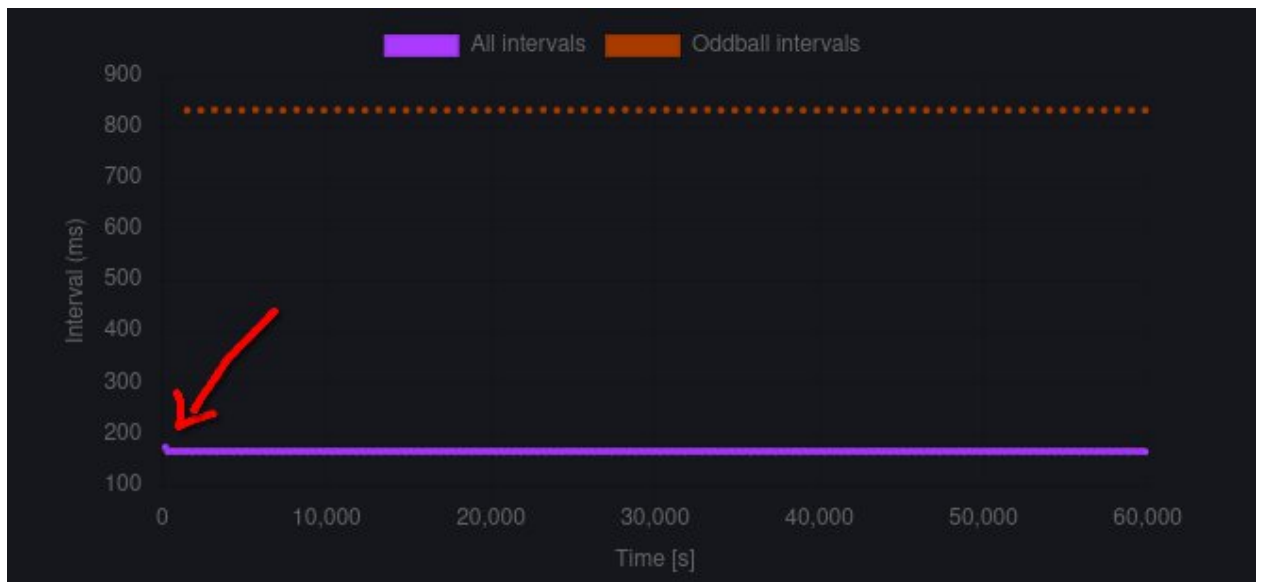


Figure 2: Stimuli interval timings. Show consistent interval timings for all the stimuli (oddball intervals displayed separately). Only the first stimulus interval is significantly higher than the rest (marked by red arrow).

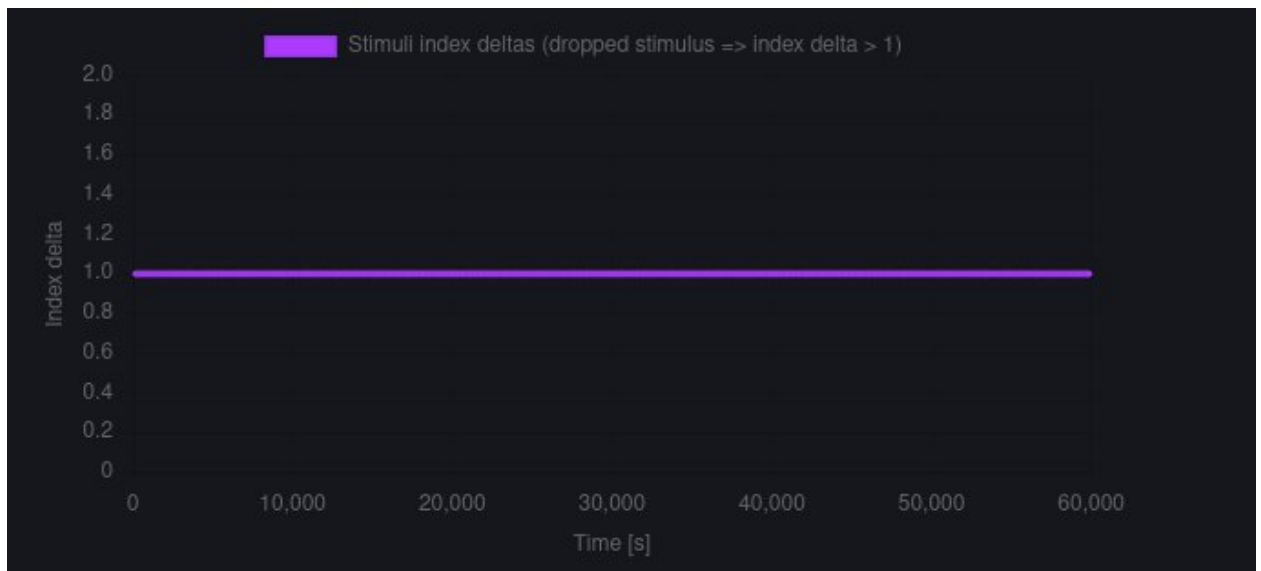


Figure 3: No dropped stimuli. All stimulus log entry have a stimulus index delta of 1 meaning no index is skipped.

Ways to improve and known limits:

- Image preloading.** When a stimuli image is first used it needs to be fetched and loaded, which creates overhead. The stimuli files are very small and there aren't many stimuli files either, so the overhead isn't big. The interval plot show this behaviour as the first interval is higher than the subsequent ones. This effect is small, but not negligible and could become a bigger problem with bigger and more stimuli files. This performance dropped could be improved by preloading the images once before starting the loop, putting them in cache.
 I consider this a 'could've, would've, should've', if this was a real world project, but for this assignment I left it out of scope.
- rAF only fire once per display refresh.** This means timing can never be finer than the screen's own refresh interval and jitter on the order of 1 frame is an inherent property.

With a base rate of 6 Hz (166.67ms) and a screen refresh of 60Hz (16.67 ms), this means theoretically exactly 10 frames can be displayed, hence the very low interval jitter. If we would use a base rate that is less dividable with the refresh rate, this jitter could increase up to 16.67 in the worst case. This is an inherent limitation that can only be solved by somehow increasing the refresh rate of the app.

- **Background tab throttling or CPU load can affect timing.** If a browser decides to throttle or even pause the activity of this webapp tab (e.g. when minimised) or when the CPU load spikes for whatever reason, this can delay a frame, or even pause the rAF entirely. These risks are harder to control, but can be reduced by good computer management (not have many tabs and other programs open) and code optimisation.
- **Back button is visible during the 'stimuli sequence'.** Yes, I am aware in an actual experiment this would be undesired design as it can draw the attention from the subject. This addition however, made debugging and testing much easier, as I was able to interrupt a run with it. A nicer way to implement this desired interrupt option is with a keyboard button press. Could've, would've, should've...
- **Limited unit testing.** Only the generateSequence() function in sequenceUtils.js has been unit tested. Ideally you want better test coverage. While I still wrote tests for this specific function is that once the sequence is running it is very difficult to verify if all the images are present in the sequence and are in the right order. As this is crucial for the interpretation of the results I took time to write basic tests for this function.

LLM usage:

When using LLMs to help with coding I either use Claude Code or Lumo.

I find using these tools are most helpful for:

- **writing documentation:** docstrings (i.e. for functions), readme files (live is too short to write README's by hand...)
- **debugging and scaffolding:** a combination of stackoverflow and LLM's is usually sufficient to solve most bugs, or to obtain generic (function) templates, for e.g. csv exporters that can then be adapted to need.
- **Linting**, though using a built-in linter in your IDE, e.g. pylinter (python), eslint (JS) has my preference.
- (for large project) **Ideation:** not applicable for relatively small tasks as these.